



PracticeDecimal

🕒 Created time	July 31, 2026 2:54 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A.R.A
🕒 Last updated time	July 31, 2026 5:32 AM

תרגול מעשי: מערכת ניהול תקציב לחברת פינטק

פיתוח פיצ'ר למערכת ארנקים דיגיטליים.

את צריכה לכתוב את השלבים הבאים בקוד #C, ולשלוף את הסינטקס הנכון והמדויק ביותר על עיבור:

1. הגדרת משתנה והשמה:

הגדירי משתנה שייצג את יתרת החשבון (Account Balance) של המשתמש.

הציבי בתוכו את הערך ההתחלתי הבא: `4500.75`.

(רמז: זכרי מהדרכה שלנו איך המהדר/קומפיילר מתייחס למספרים עשרוניים כברירת מחדל ומה את חייבת להוסיף כדי שזה יעבוד בלי שגיאות המרה).

2. הסבר בקוד (הערה - Comment):

כתבי הערה קצרה מעל המשתנה, שבה תסבירי במשפט אחד או שניים: למה בחרת בסוג המשתנה הזה ספציפית ולא ב- `Double`? (התייחסי בהסבר שלך לדיוק מול עלות של זיכרון).

3. פעולה חשבונית בסיסית:

המשתמש כרגע קיבל זיכוי. הוסיפי ליתרת החשבון שלו את הסכום: `150.50`.

4. שאלת הבנה (הערה בסוף הקוד):

המנהל שלך מבקש שתוסיפי פונקציה שמחשבת שורש ריבועי (Square Root) על יתרת החשבון של המשתמש בעזרת ספריות המתמטיקה המובנות של מיקרוסופט.

כתבי הערה קצרה בקוד שמסבירה למנהל שלך למה זה בעייתי עם סוג המשתנה שבחרת לעבוד איתו לפיתוח הזה.

```
//1
//Variable definition and naming:
```

```
decimalaccountBalance = 4500.75M;
```

//~~~~~

```
//2

//Why Decimal and not Double?

//I chose Decimal because dealing with money requires ultra precision.

//However, Decimal consumes much more memory space than Double (a higher memory cost).

//Therefore, it's a cost benefit analysis: we only pay this expensive memory price when
```

```
absolute precision is required, like with
//money.
```

//~~~~~

```
//3
//The user has just received a credit.
//Add the amount to their account balance: 150.50.

accountBalance += 150.50M;

Console.WriteLine(accountBalance);
```

//~~~~~

```
//4
//Your manager wants you to add a function that calculates the square root of a user's
account balance using Microsoft's built in math libraries.
//Write a short note in the code explaining to your manager why this is problematic with
the variable type you chose to work with for this development.
//Advanced functions are not designed to work directly on `Decimals`.
//It is much cheaper and more efficient to calculate operations that return `Doubles`
//All of Microsoft's built-in math functions, such as the `Math` class, work with `Doubles`
and not with `Decimals`.
//We will only do this with money
//Otherwise it is simply too computationally expensive, so for most math needs we use `
Double`.
```

//~~~~~

תרחיש התרגול: מערכת הנהלת חשבונות ב-FinTech

משימה 1: הצהרה

צרי משתנה שנועד להחזיק יתרת עובר ושב של לקוח.

זכרי באיזה **Type** נתונים עלינו להשתמש כאשר אנחנו עובדים עם כסף וצריכים דיוק מקסימלי (עד 28 מקומות עשרוניים), למרות שהוא דורש הקצאה גדולה יותר בזיכרון בהשוואה לטיפוסים אחרים.

משימה 2: אתחול

הציבי במשתנה שיצרת את הערך: `10500.50`.

דגש כתיבה: זכרי להשתמש באות (Suffix) המדויקת בסוף המספר.

אחרת, הקומפילר יזהה את המספר כ-Double כברירת מחדל.

משימה 3: פעולות חשבון

הלקוח ביצע הפקדה של `200.75` ולאחר מכן חויב בעמלה של `1.50`.

עדכני את המשתנה בעזרת פעולות חשבון (חיבור וחסור - הפעולות הזולות והנפוצות ביותר לטיפול נתונים זה, בשונה מכפל וחילוק). לבסוף, הדפיסי את התוצאה לקונסול.

משימה 4: הבנת הלוגיקה והשגיאות

צרי משתנה חדש מאותו **Type** נתונים (של כסף), וקראי לו `coffeePrice`.

הציבי בו את הערך 12.5 ללא האות המזהה בסוף המספר.
הקומפילר יסמן לך שגיאה.

מתחת לשורת הקוד הזו, כתבי לי בהערה הסבר מפורט אך קצר: מדוע אי אפשר לעשות פה Implicit Conversion מ-Double?
דגש להסבר שלך: התייחסי למושגים של רמת דיוק (15-16 מקומות לעומת 28 מקומות) ולסכנה של הכנסת Bad Data.

```
//1
//Create a variable to hold a customer's current account balance.
//Remember which Type data type we should use when we work with money and need maximum
precision (up to 28 decimal places), even though it requires a larger memory allocation
compared to other types.

decimal customerAccountBalance;
```

//~~~~~

```
//2
//Set the variable you created to the value: `10500.50`.
//Writing emphasis: Remember to use the exact suffix at the end of the number.
//Otherwise, the compiler will recognize the number as Double by default.

customerAccountBalance = 10500.50M;
```

//~~~~~

```
//3
//The customer made a deposit of `200.75` and was then charged a fee of `1.50`.
//Update the variable using arithmetic operations (addition and subtraction - the cheap
est and most common operations for this data type, unlike multiplication and division).
Finally, print the result to the console.

customerAccountBalance += 200.75M;

customerAccountBalance -= 1.50M;

Console.WriteLine(customerAccountBalance);
```

//~~~~~

```
//4
//Create a new variable of the same Type data (of money), and call it `coffeePrice`.
//Set it to the value `12.5` without the identifying letter at the end of the number.
//The compiler will flag you as an error.
//Under this line of code, write me a detailed but short explanation in a comment: Why
is it impossible to do an Implicit Conversion from Double here?
//Emphasis on your explanation: Refer to the concepts of precision (15-16 places compar
ed to 28 places) and the danger of inserting Bad Data.
```

```
//decimal coffeePrice = 12.5;
```

```
//Explanation:
```

```
//It is not possible to perform an implicit conversion from Double to Decimal due to the danger of introducing Bad Data.
```

```
//The Double type holds numbers with a precision of 15-16 decimal places and is rounded, while Decimal maintains absolute precision up to 28 decimal places.
```

```
//If we insert a less precise value (Double) into a variable that expects maximum precision (Decimal), we will actually insert rounded and unreliable information into it.
```

```
//Since Decimal is intended for money, the compiler protects us and prevents us from polluting the variable with data that may be inaccurate.
```